

MoRE OS Import Task Document Specification

Version: 1.0.0

Status: DRAFT

Author: MoRE OS Core Team

Date: 2026-07-22

1. Overview

The Import Task Document (ITD) is a rigorous Markdown format for defining executable tasks in MoRE OS. ITD files serve as the **single source of truth** for task definition, allowing humans and AI agents to author, review, validate, and import tasks with full structural fidelity.

1.1 Design Principles

Principle	Rationale
Human-readable first	Markdown with YAML frontmatter — editable in any text editor, reviewable in any Markdown viewer
Machine-parseable	Strict section hierarchy + frontmatter enables lossless round-trip parsing
Composable	ITD files can reference other ITDs via <code>depends_on</code> , enabling DAG-style task graphs
Rigorous by default	Every task must define a deliverable contract and acceptance criteria; kill criteria are recommended
Pipeline-aware	Pipeline hints in frontmatter let domain experts guide routing without modifying core config

1.2 File Extension and Naming

```
<project>-<module>-<seq>.task.md
```

Examples: - more-core-import-task-001.task.md - app-auth-flow-002.task.md

2. Document Structure

An ITD file consists of two parts:

```
+-- YAML Frontmatter (delimited by ---) --+
|                                           |
|  Metadata, routing hints, budget         |
|                                           |
+-----+
|                                           |
|  Markdown Body (rigid section headings)  |
|                                           |
|  1. Executive Summary                   |
|  2. Requirements                       |
|  3. Deliverable Contract               |
|  4. Kill Criteria (optional)           |
|  5. Resource Budget                   |
|  6. Context & Constraints               |
|  7. Related Documents (optional)       |
|                                           |
+-----+
```

3. YAML Frontmatter Schema

```
---
title: string                # Required. Human-readable task title.
version: semver              # Required. Document version (e.g. "1.0.0").
author: string               # Required. Author name or team.
created: ISO8601             # Required. Creation date (e.g. "2026-07-22T10:00:00Z").
type: TaskType               # Required. One of the MoRE TaskType enum values:
                             #   code_generation | code_debugging | code_review
                             #   code_testing | math_reasoning | data_analysis
                             #   nlp_task | architecture_design | multi_agent
                             #   self_improvement | cross_domain | plugin_defined
plugin_type: string | null   # Plugin identifier when type=plugin_defined.
priority: Priority           # Required. high | medium | low.
deliverable_kind: Kind       # Required. code | architecture | analysis | decision
                             #   | plan | explanation | creative | custom.
tags: list[string]          # Required. At least one tag.
estimated_hours: float       # Optional. Time estimate in hours.
depends_on: list[string]      # Optional. List of task IDs this task depends on.
pipeline:                   # Optional. Pipeline override hints.
```

```

mode: quick | standard | deep # Routing mode hint.
prepend: list[LayerId] # Layers to prepend to default pipeline.
append: list[LayerId] # Layers to append to default pipeline.
skip: list[LayerId] # Layers to skip in default pipeline.
target_confidence: float # Optional. 0.0-100.0. Default 60.0.
max_iterations: int # Optional. Default 10.
timeout_s: float # Optional. Default 60.0.
allow_self_improvement: bool # Optional. Default false.
require_metacognitive: bool # Optional. Default false.
kill_on_diverge: bool # Optional. Default true.
---
```

4. Markdown Body Sections

4.1 Executive Summary

Heading: # Executive Summary

A concise description of the task goal, motivation, and expected outcome. 1–3 paragraphs.

```
# Executive Summary
```

Implement a token-aware rate limiter for the LLM provider chain. Currently the system has no per-provider rate limiting, causing 429 errors from upstream APIs under burst load. This task adds a sliding-window rate limiter with per-provider configuration, fallback queuing, and Prometheus metrics export.

4.2 Requirements

Heading: # Requirements

A list of formal requirement items. Each item has a unique ID (REQ-NNN), title, description, acceptance criteria, and priority.

```
# Requirements
```

```
## REQ-001: Sliding Window Implementation
```

```
**Priority:** HIGH
```

```
**Description:** Implement a sliding-window rate limiter that tracks requests per
provider per time window. Window size and max request count must be configurable
per provider.
```

```
**Acceptance Criteria:**
```

- [] Rate limiter rejects requests exceeding `max\_requests` per `window\_seconds`
- [] Window slides correctly after idle periods
- [] Config is reloadable without service restart
- [] All edge cases documented (zero window, zero max, concurrent access)

```
## REQ-002: Provider Configuration
```

```

**Priority:** MEDIUM
**Description:** Each LLM provider must expose rate limit settings in its config,
with sensible defaults (LM Studio: 30 req/min, Ollama: 60 req/min, DeepSeek: 3000 req/min).

**Acceptance Criteria:**
- [ ] Default config values for all 3 providers
- [ ] Config overridable via environment variable and .env
- [ ] Configuration validated at startup with clear error messages

```

4.3 Deliverable Contract

Heading: # Deliverable Contract

Defines what "done" means. Includes required output dimensions, quality gates, and formal acceptance criteria.

```

# Deliverable Contract

**Kind:** code
**Required Dimensions:** core_output, reasoning, tests, performance

**Quality Gates:**
| Gate | Threshold |
|-----|-----|
| min_output_length | 200 characters |
| must_contain_code_fence | true |
| must_contain_test_cases | true |
| test_coverage_pct | >= 80 |

**Acceptance Criteria:**
- [ ] All rate limiter unit tests pass with >80% coverage
- [ ] Integration test with mocked providers confirms fallback behavior
- [ ] Benchmark shows <1ms overhead per rate-limited call
- [ ] Documentation updated: rate limiting section in OPERATION_MANUAL.md

```

Reserved Acceptance Criterion IDs

Each acceptance criterion automatically receives a machine-readable ID:

```

AC-001: All rate limiter unit tests pass with >80% coverage
AC-002: Integration test with mocked providers confirms fallback behavior
AC-003: Benchmark shows <1ms overhead per rate-limited call
AC-004: Documentation updated: rate limiting section in OPERATION_MANUAL.md

```

4.4 Kill Criteria

Heading: # Kill Criteria

Conditions that should cause task termination. Optional but recommended for complex tasks.

```
# Kill Criteria
```

```
| ID | Condition | Severity | Timeline | Fallback |
|----|-----|-----|-----|-----|
| KC-001 | Test coverage drops below 70% | critical | immediate | Skip tests, file issue |
| KC-002 | Benchmark overhead exceeds 5ms | warning | end-of-run | Log warning, proceed with d
| KC-003 | API-breaking change required | fatal | immediate | Block, require ADR |
```

Each criterion maps to the `KillCriterion` dataclass in `core/deliverable.py`:-

condition: human-readable trigger description - **severity:** `fatal` | `critical` | `warning` -

timeline: `immediate` | `end-of-run` - **fallback:** action to take if triggered - **trigger:**

(optional) machine-readable predicate expression

4.5 Resource Budget

Heading: `# Resource Budget`

Token, time, and compute budget specifications.

```
# Resource Budget
```

```
**Estimated Tokens:** 12,000
**Estimated Duration:** 30 minutes
**Max Iterations:** 5
```

```
**Per-Provider Token Budget:**
```

```
| Provider | Input | Output | Total |
|-----|-----|-----|-----|
| lmstudio | 4,000 | 8,000 | 12,000 |
| ollama | 2,000 | 4,000 | 6,000 (fallback only) |
```

```
**Rate Limit Overrides:**
```

```
| Provider | Context Window | Max Tokens |
|-----|-----|-----|
| lmstudio | 8,192 | 2,048 |
| deepseek | 32,768 | 4,096 |
```

4.6 Context and Constraints

Heading: `# Context and Constraints`

Background information, technical constraints, architectural decisions, and references.

```
# Context and Constraints
```

```
## Background
```

The current LLM manager (`llm/manager.py`) has a single global rate limit that does not distinguish between providers. This causes DeepSeek API calls to be unnecessarily throttled by LM Studio's conservative limits.

```

## Constraints
- Must not introduce new external dependencies
- Must maintain backward compatibility with existing provider configs
- Rate limiter must be debuggable: expose stats endpoint

## References
- [llm/manager.py](../more_core/more_core/llm/manager.py)
- [core/config.py](../more_core/more_core/core/config.py)
- ADR-0014: LLM Provider Architecture

```

4.7 Related Documents

Heading: # Related Documents

Links to other task documents, ADRs, or external references.

```

# Related Documents

- [MORE-002: Provider Health Check](more-002-provider-health.task.md)
- [ADR-0014: LLM Provider Architecture](../docs/adr/ADR-0014-llm-provider.md)

```

5. Complete Example

```

---
title: Token-Aware Rate Limiter for LLM Providers
version: 1.0.0
author: MoRE OS Core Team
created: 2026-07-22T10:00:00Z
type: code_generation
priority: high
deliverable_kind: code
tags: [llm, rate-limiting, reliability]
estimated_hours: 8.0
depends_on: []
pipeline:
  mode: standard
  target_confidence: 80.0
  max_iterations: 5
  timeout_s: 120.0
  allow_self_improvement: false
  require_metacognitive: false
  kill_on_diverge: true
---

# Executive Summary

Implement a sliding-window rate limiter for the LLM provider chain to prevent
429 errors under burst load. Each provider gets independent rate limit config
with sensible defaults. Includes Prometheus metrics export.

# Requirements

```

```

## REQ-001: Sliding Window Implementation
**Priority:** HIGH
**Description:** Implement sliding-window rate limiter with per-provider config.

**Acceptance Criteria:**
- [ ] Rejects requests exceeding max_requests per window_seconds
- [ ] Window slides correctly after idle periods
- [ ] Config reloadable without restart

## REQ-002: Provider Defaults
**Priority:** MEDIUM
**Description:** Sensible defaults for all three providers.

**Acceptance Criteria:**
- [ ] LM Studio: 30 req/min, Ollama: 60 req/min, DeepSeek: 3000 req/min
- [ ] Config overridable via env vars
- [ ] Validation at startup

# Deliverable Contract

**Kind:** code
**Required Dimensions:** core_output, reasoning, tests
**Minimum Output Length:** 200

**Quality Gates:**
| Gate | Threshold |
|-----|-----|
| min_output_length | 200 |
| must_contain_code_fence | true |
| test_coverage_pct | >= 80 |

**Acceptance Criteria:**
- [ ] Unit tests pass with >80% coverage
- [ ] Integration test confirms fallback behavior
- [ ] Benchmark shows <1ms overhead

# Kill Criteria

| ID | Condition | Severity | Timeline | Fallback |
|----|-----|-----|-----|-----|
| KC-001 | Coverage < 70% | critical | immediate | Skip tests, file issue |
| KC-002 | Overhead > 5ms | warning | end-of-run | Log warning, proceed |
| KC-003 | API-breaking change | fatal | immediate | Block, require ADR |

# Resource Budget

**Estimated Tokens:** 12,000
**Estimated Duration:** 30 minutes

| Provider | Input | Output | Total |
|-----|-----|-----|-----|
| lmstudio | 4,000 | 8,000 | 12,000 |
| ollama | 2,000 | 4,000 | 6,000 |

# Context and Constraints

```

```

## Background
Current llm/manager.py has single global rate limit. DeepSeek calls throttled
by LM Studio's conservative limits.

## Constraints
- No new external dependencies
- Backward compatible with existing configs
- Stats endpoint for debugging

## References
- llm/manager.py
- core/config.py

```

6. Validation Rules

When an ITD is imported, the parser validates:

Rule	Check	Error Type
R001	Frontmatter is valid YAML	fatal
R002	<code>title</code> is present and non-empty	fatal
R003	<code>type</code> is a valid TaskType	fatal
R004	<code>priority</code> is high/medium/low	fatal
R005	<code>deliverable_kind</code> is a valid kind	fatal
R006	<code>tags</code> is a non-empty list	warning
R007	<code>Executive Summary</code> section exists	warning
R008	<code>Deliverable Contract</code> section exists	warning
R009	At least one acceptance criterion present	warning
R010	<code>pipeline.mode</code> is quick/standard/deep if specified	warning
R011	All REQ-XXX IDs are unique	fatal
R012	All AC-XXX references resolve	warning
R013	<code>depends_on</code> references are well-formed	warning
R014	Resource budget is internally consistent	warning

7. Pipeline Integration

When an ITD is imported via `POST /api/v1/tasks/import`, the system:

1. **Parse** the Markdown into structured `ImportTaskDocument`
 2. **Validate** against all rules (Section 6)
 3. **Transform** into a `TaskRequest`:
 4. `type` $\leftarrow$ `frontmatter type`
 5. `query` $\leftarrow$ Executive Summary
 6. `deliverable_kind` $\leftarrow$ `frontmatter deliverable_kind`
 7. `expectation` $\leftarrow$ build from Deliverable Contract + Kill Criteria
 8. `timeout_s` $\leftarrow$ `frontmatter timeout_s`
 9. `allow_self_improvement` $\leftarrow$ `frontmatter allow_self_improvement`
 10. `require_metacognitive_monitoring` $\leftarrow$ `frontmatter require_metacognitive`
 11. `context` $\leftarrow$ full parsed document (for reference layers)
 12. **Route** through the layer pipeline with ITD hints
 13. **Execute** and produce `TaskResult`
 14. **Store** the result with reference to the source ITD
-

8. Serialization

8.1 to_dict()

Every ITD document can be serialized to a Python dict for API transport:

```
{
  "metadata": { ... frontmatter fields ... },
  "summary": "text",
  "requirements": [
    {"id": "REQ-001", "title": "...", "description": "...",
     "priority": "HIGH", "acceptance_criteria": ["...", "..."]}
  ],
  "deliverable_contract": {
    "kind": "code",
    "required_dimensions": ["core_output", "reasoning", "tests"],
    "quality_gates": {"min_output_length": 200, "must_contain_code_fence": True},
    "acceptance_criteria": ["Unit tests pass", "Integration test confirms"]
  },
  "kill_criteria": [
    {"id": "KC-001", "condition": "...", "severity": "critical", "timeline": "immediate",
    },
  ],
  "resource_budget": {
    "estimated_tokens": 12000,
    "estimated_duration_min": 30,
  },
}
```

```

    "per_provider": {"lmstudio": {"input": 4000, "output": 8000}}
  },
  "context": {"background": "...", "constraints": ["...", "..."], "references": ["..."]},
  "related_documents": ["..."]
}

```

8.2 to_markdown()

Generate a well-formatted ITD Markdown string from a structured object. This enables: -
 Programmatic task generation - Template instantiation - Round-trip editing

8.3 to_yaml()

Export only the frontmatter + contract as a compact YAML snippet for use in CI/CD pipelines.

9. Templates

9.1 Minimal Template

```

---
title: "{{ title }}"
version: 1.0.0
author: "{{ author }}"
created: "{{ created }}"
type: code_generation
priority: medium
deliverable_kind: code
tags: []
---

# Executive Summary

{{ summary }}

# Requirements

## REQ-001: {{ requirement_title }}
**Priority:** MEDIUM
**Description:** {{ description }}

**Acceptance Criteria:**
- [ ] {{ criterion_1 }}
- [ ] {{ criterion_2 }}

# Deliverable Contract

**Kind:** code
**Required Dimensions:** core_output, reasoning
**Minimum Output Length:** 100

```

```

**Quality Gates:**
| Gate | Threshold |
|-----|-----|
| min_output_length | 100 |

**Acceptance Criteria:**
- [ ] {{ criterion_1 }}
- [ ] {{ criterion_2 }}

# Resource Budget

**Estimated Tokens:** 4,000
**Estimated Duration:** 15 minutes

```

9.2 Architecture Design Template

```

---
title: "{{ title }}"
version: 1.0.0
author: "{{ author }}"
created: "{{ created }}"
type: architecture_design
priority: high
deliverable_kind: architecture
tags: [architecture]
---

# Executive Summary

{{ summary }}

# Requirements

## REQ-001: {{ requirement_title }}
**Priority:** HIGH
**Description:** {{ description }}

**Acceptance Criteria:**
- [ ] {{ criterion }}

# Deliverable Contract

**Kind:** architecture
**Required Dimensions:** core_output, reasoning, risks, alternatives
**Minimum Output Length:** 200

**Quality Gates:**
| Gate | Threshold |
|-----|-----|
| min_output_length | 200 |
| must_contain_modules | true |
| must_contain_interfaces | true |

**Acceptance Criteria:**

```

- [] Architecture diagram included
- [] At least 2 alternatives evaluated
- [] Risk assessment with mitigation plan

Kill Criteria

ID	Condition	Severity	Timeline	Fallback
KC-001	No viable alternative found	fatal	immediate	Escalate to L5

Resource Budget

****Estimated Tokens:** 8,000**
****Estimated Duration:** 60 minutes**
****Max Iterations:** 3**